Assignment - 2

Name : Prachiti Suryawanshi
PRN : 12622521 77
Class : SY BTech CSE-CSF

Aim : Write a C program for sparse matrix realization & operations on it
→ Transpose → Fast transpose

Objective :
- To study the concept of sparse matrix, how it is stored & displayed.
- To understand the concept of implementation of sparse matrix operations.

Platform :
64-Bit open source linux
Open source C programming tool

Pseudo Code :

Step 1 - START
Step 2 - Read rows & columns
k ← 1
for i=0 to rows -1
  for j=0 to columns -1

    Read matrix [i][j]

Sundaram
FOR EDUCATIONAL USE

```
        if matrix[i][j] ≠ 0
            compact[k].row = i
            compact[k].col = j
            compact[k].value = matrix[i][j]

            k = k+1
        End if
    End for
End for
```

Step 3 - Transpose.[0].row = original.columns
Transpose.[0].col = original.rows
Transpose.[0].value = original.non-zero

```
k ← 1
    If matrix[i][j] ≠ 0
        compact[k].row = i
        compact[k].col = j
        compact[k].value = matrix[i][j]
    k = k+1
```

Step 4 - Initialize Total[ ] = 0

```
for each non zero element
    Total[column]++
end for

INDEX[0] = 1
```

FOR EDUCATIONAL USE

Sundaram

```
for i = 1 to column - 1
        INDEX[i] = INDEX[i-1] + Total[i-1]
end for
for each non zero element.
        position = INDEX[element.column]
end for
```

Step 5 - STOP

Time Complexity :

Operation	Time complexity
Compact representation	O (m x n)
Simple Transpose.	O (col. x non zero elements)
Fast Transpose	O (col. + non zero elements)

Conclusion : Thus implemented sparse matrix operation. This system is able to perform diff. operations. on sparse matrix such as simple & fast transpose & their time complexity.

Sundaram
FOR EDUCATIONAL USE

FAQs:

1) What is a sparse matrix? List the applications.

Sparse matrix has most of the elements as zero [0] & only a few elements as non zero.

eg.
$$\begin{matrix} 0 & 0 & 3 & 0 & 4 \\ 0 & 0 & 5 & 7 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 6 & 0 & 0 \end{matrix} \quad (4 \times 5)$$

Applications: Graph representation
Image processing
Compute graphics
Network analysis
Database system

2) Represent sparse matrix with suitable data structure. Explain simple & fast transpose with an example.

Representation of SM:

- Triplet (compact) Rep. ← Most common

each non zero member is stored as (Row, Column, Value).

Sundaram
FOR EDUCATIONAL USE

First row →
(no. of rows, no. of columns, no. of non-zero ele.)

eg.

0	0	3	0	4
0	0	5	7	0
0	0	0	0	0
0	6	2	0	0

Row	Column	Value
4	5	6
0	2	3
0	4	4
1	2	3
1	3	7
3	1	2
3	2	6

Simple Transpose :
Converts rows into columns by scanning one by one columns.

Algorithm :
1) Copy header row
2) For each column
   → scan all non zero elements
   → when column matches, interchanges rows & columns.

Sundaram
FOR EDUCATIONAL USE

eg.

Original			ST		
Row	Column	Value	Row	Column	Value
4	5	6	5	4	6
0	2	3	1	3	2
0	4	4	2	0	3
1	2	5	2	1	5
1	3	7	2	3	6
3	1	2	3	1	7
3	2	6	4	0	4

Time complexity :
O(columns x Non zero elements)

Fast Transpose :
Improves efficiency by using more arrays.
- Total[ ] - stores non zero element
- Index[ ] - stores starting position

Algorithm :
1) Copy the header
2) Count non zero element using Total[ ]
3) Compute starting position using Index[ ]
4) Place every element directly into its convert position
5) Increment corresponding index

Sundaram
FOR EDUCATIONAL USE

eg.

Total [ ]

column	0	1	2	3	4
count	0	1	3	1	1

Index [ ]

column	0	1	2	3	4
start pos.	1	1	2	5	6

Final :

Row	Column	Value
5	4	6
1	3	2
2	0	3
2	1	5
2	3	6
3	1	7
4	0	1

Time complexity : O(columns + non-zero elements)

3) Find out the addition of 2 sparse matrix in triplet form & also find simple & fast transpose.

M1 :

4	5	6
0	3	5
1	3	6
1	4	45
2	3	4
3	2	45
4	1	2

M2 :

4	5	6
0	3	7
0	4	6
1	4	4
2	1	8
3	2	45
4	4	21

FOR EDUCATIONAL USE

Addition :

Position	M1	M2	Sum
(0,3)	5	7	12
(0,4)	0	6	6
(1,3)	8	0	8
(1,4)	45	4	49
(2,1)	0	8	8
(2,3)	4	0	4
(3,2)	45	45	90
(4,1)	2	0	2
(4,4)	0	21	21

Total non-zero element = 9

Result & triplet form

Row	Column	Value
4	5	9
0	3	12
0	4	6
1	3	8
1	4	49
2	1	8
2	3	4
3	2	90
4	1	2
4	4	2

Simple transpose

Row	Column	Value
5	4	6
1	4	2
2	3	40
3	0	5
3	1	8
3	2	4
4	1	45

Sundaram
FOR EDUCATIONAL USE

Fast Transpose :

Row	Column	Value
5	4	6
1	4	2
2	3	45
3	0	5
3	1	8
3	2	4
4	1	45

Total[ ]

Column	0	1	2	3	4
Count	0	1	1	3	1

Sundaram
FOR EDUCATIONAL USE

```c
#include <stdio.h>

#define MAX 100

void display(int sparse[][3], int terms)
{
    int i;

    printf("\nRow\tCol\tValue\n");

    for(i = 0; i < terms; i++)
    {
        printf("%d\t%d\t%d\n", sparse[i][0], sparse[i][1], sparse[i][2]);
    }
}

void simpleTranspose(int sparse[][3], int transpose[][3])
{
    int i, j, k = 1;

    int rows = sparse[0][0];
    int cols = sparse[0][1];
```

```
23      int terms = sparse[0][2];
24
25      transpose[0][0] = cols;
26      transpose[0][1] = rows;
27      transpose[0][2] = terms;
28
29      for(i = 0; i < cols; i++)
30      {
31          for(j = 1; j <= terms; j++)
32          {
33              if(sparse[j][1] == i)
34              {
35                  transpose[k][0] = sparse[j][1];
36                  transpose[k][1] = sparse[j][0];
37                  transpose[k][2] = sparse[j][2];
38                  k++;
39              }
40          }
41      }
42  }
```

```
43
44  void fastTranspose(int sparse[][3], int transpose[][3])
45  {
46      int rowTerms[MAX], startPos[MAX];
47      int i, j;
48
49      int rows = sparse[0][0];
50      int cols = sparse[0][1];
51      int terms = sparse[0][2];
52
53      transpose[0][0] = cols;
54      transpose[0][1] = rows;
55      transpose[0][2] = terms;
56
57      if(terms > 0)
58      {
59          for(i = 0; i < cols; i++)
60              rowTerms[i] = 0;
61
62          for(i = 1; i <= terms; i++)
63              rowTerms[sparse[i][1]]++;
```

```
63              rowTerms[sparse[i][1]]++;
64
65          startPos[0] = 1;
66
67          for(i = 1; i < cols; i++)
68              startPos[i] = startPos[i-1] + rowTerms[i-1];
69
70          for(i = 1; i <= terms; i++)
71          {
72              j = startPos[sparse[i][1]];
73
74              transpose[j][0] = sparse[i][1];
75              transpose[j][1] = sparse[i][0];
76              transpose[j][2] = sparse[i][2];
77
78              startPos[sparse[i][1]]++;
79          }
80      }
81  }
82
83  int main()
```

```c
{
    int matrix[10][10];
    int sparse[MAX][3], transpose[MAX][3], fastT[MAX][3];

    int rows, cols;
    int i, j, k = 1;

    printf("Enter number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    printf("Enter matrix elements:\n");

    for(i = 0; i < rows; i++)
    {
        for(j = 0; j < cols; j++)
        {
            scanf("%d", &matrix[i][j]);

            if(matrix[i][j] != 0)
            {
                sparse[k][0] = i;
                sparse[k][1] = j;
```

```
106                 sparse[k][2] = matrix[i][j];
107                 k++;
line C Compiler         }
109             }
110         }
111
112         sparse[0][0] = rows;
113         sparse[0][1] = cols;
114         sparse[0][2] = k - 1;
115
116         printf("\nSparse Matrix (Triplet Form):");
117         display(sparse, k);
118
119         simpleTranspose(sparse, transpose);
120
121         printf("\nSimple Transpose:");
122         display(transpose, k);
123
124         fastTranspose(sparse, fastT);
125
126         printf("\nFast Transpose:");
```

```
126     printf("\nFast Transpose:");
127     display(fastT, k);
128
129     return 0;
130 }
```

Output

```
Enter number of rows and columns: 2 2
Enter matrix elements:
1 2
3 4

Sparse Matrix (Triplet Form):
Row Col Value
2   2   4
0   0   1
0   1   2
1   0   3
1   1   4

Simple Transpose:
Row Col Value
2   2   4
0   0   1
0   1   3
1   0   2
1   1   4
```

```
Fast Transpose:
Row  Col  Value
2    2    4
0    0    1
0    1    3
1    0    2
1    1    4
```